

Informe Técnico

Sistema de Créditos IPSFA

Arquitectura, base de datos, API, seguridad, refactorización y estado operativo del sistema

Versión del sistema	v6.1 (Refactorización + Seguridad)
Fecha de análisis	19 de julio de 2026
Alcance	Código fuente, base de datos y despliegue
Metodología	Análisis estático completo del código fuente y del esquema de base de datos
Clasificación	Documento interno — uso técnico

Índice

1. Resumen ejecutivo	3
2. Arquitectura general	5
2.1 Pila tecnológica	5
2.2 Diagrama de arquitectura	6
3. Estructura del proyecto	7
4. Base de datos	8
4.1 Tablas del sistema	8
4.2 Índices, triggers y auditoría	9
4.3 Capa de estadísticas (no instalada)	9
4.4 Scripts de inicialización y carga	10
5. API REST	11
6. Modelo de seguridad (v6.1)	15
6.1 Autenticación y revocación de sesiones	15
6.2 Control de acceso por roles y tienda	15
6.3 Protecciones de red y aplicación	16
7. Frontend	17
8. Refactorización v6: antes y después	18
9. Estado real del módulo de estadísticas	19
10. Despliegue y operación	20
11. Hallazgos y recomendaciones	22
12. Anexos	24

1. Resumen ejecutivo

El **Sistema de Créditos IPSFA** es una aplicación web interna para la gestión de créditos por tienda (Caracas, Maracay y Maracaibo): registro de facturas a crédito, control de once cuotas mensuales por factura, tasas de cambio BCV, agenda de actividades, reportes operativos y administración de usuarios con dos roles (administrador y operador). El sistema opera en una red local, con backend Node.js/Express, base de datos PostgreSQL y un frontend de aplicación de página única (SPA) sin frameworks.

La versión analizada, **v6.1**, incorpora dos esfuerzos de ingeniería recientes: la **refactorización v6**, que eliminó la triplicación de código de tiendas consolidándola en un módulo genérico configurable, y el **paquete de seguridad v6.1**, que cerró seis vulnerabilidades críticas y seis altas (endpoints sin autenticación, fuga de tokens de recuperación, suplantación de restricción por IP, entre otras) e introdujo revocación de sesiones mediante `token_version`.

Tabla 1-1. Ficha técnica del sistema (v6.1)

Métrica	Valor
Líneas de código (sin dependencias)	14.113 líneas en 16 archivos fuente propios
Backend	Node.js + Express 4 — 2.315 líneas (servidor, configuración y middleware)
Rutas API	7 módulos, 37 endpoints REST (2.092 líneas)
Frontend	SPA en JavaScript puro — 6.172 líneas (HTML + JS) y 4.778 líneas CSS
Base de datos	PostgreSQL — 8 tablas, 8 secuencias, 19 índices, 2 triggers
Tiendas soportadas	3 (Caracas, Maracay, Maracaibo) mediante módulo genérico configurable
Roles de usuario	2 (administrador, operador con alcance por tienda)
Autenticación	JWT (8 h) con revocación por <code>token_version</code> ; contraseñas bcrypt (12 rondas)
Despliegue actual	PC Windows 11, ejecución manual en terminal (<code>node server.js</code>)

1.1 Estado general del sistema

El sistema se encuentra **funcional y en producción** para su flujo principal (login, panel, gestión de las tres tiendas, tasas BCV, actividades y reportes). El análisis estático completo realizado para este informe — que por primera vez incluye el esquema real de la base de datos y el árbol de archivos del proyecto — reveló, sin embargo, un conjunto de hallazgos que conviene atender, resumidos a continuación y desarrollados en la [sección 11](#):

- **Módulo de estadísticas no instalado.** Las cuatro funciones SQL y la vista que alimentan `/api/estadisticas/*` existen como script (`database/estadisticas.sql`) pero nunca fueron ejecutadas en la base de datos; la página `estadisticas.html` tampoco existe, por lo que la ruta `/estadisticas` está rota. El módulo está desplegado a medias.

- **Tabla `sesiones` sin uso.** Existe en la base de datos con índices propios, pero ningún archivo del sistema la lee ni la escribe (la revocación v6.1 se resolvió con `token_version`).
- **Inconsistencias menores de esquema entre tiendas** (columna `created_at` ausente en `tienda_maracaibo` ; trigger de `updated_at` presente solo en Maracay; cobertura de índices desigual).
- **Despliegue manual.** El servidor vive en una ventana de terminal: si la PC se reinicia o la ventana se cierra, el sistema queda fuera de servicio hasta intervención manual.
- **Archivos legacy** de versiones anteriores aún presentes en `public/` (páginas independientes de tienda y reportes), uno de ellos con referencia rota.

Lectura rápida. Quien busque una visión global puede leer las secciones 1, 2 y 11. El detalle de base de datos está en la sección 4; el de la API, en la 5; el modelo de seguridad, en la 6; y la operación diaria, en la 10.

2. Arquitectura general

El sistema sigue una arquitectura **cliente-servidor de tres capas** clásica: el navegador del usuario ejecuta el frontend estático (HTML/CSS/JS servido por el propio Express), el backend expone una API REST JSON bajo `/api/*`, y PostgreSQL aloja tanto los datos como —potencialmente— parte de la lógica de estadísticas en funciones SQL. No hay balanceadores, cachés intermedios ni colas: es un monolito modular, apropiado para su escala (una oficina, pocas sesiones concurrentes).

2.1 Pila tecnológica

Tabla 2-1. Componentes y versiones

Capa	Tecnología	Detalle
Backend	Node.js + Express	Express 4.x; servidor en <code>server.js</code> (124 líneas tras refactorización)
Base de datos	PostgreSQL	14+; acceso mediante <code>pg</code> (Pool de conexiones) 8.x
Autenticación	jsonwebtoken 9.0.2	JWT HS256, expiración 8 h, claim <code>tv</code> para revocación
Hashing	bcryptjs ^3.0.2	12 rondas de sal
Limitación de tasa	express-rate-limit ^7.4.0	Protección de fuerza bruta en <code>/api/auth/login</code>
Red	cors 2.8.5, dotenv 16.3.1	CORS con lista blanca por entorno; configuración por variables
Frontend	HTML5 + CSS3 + JS puro	Sin framework ni build; SPA montada sobre <code>panel.html</code>
Gráficos (CDN)	ApexCharts 3.45.1	Gráficos del panel y reportes
Exportación (CDN)	jsPDF 2.5.1, SheetJS xlsx 0.18.5	Reportes PDF y Excel generados en el navegador
UI (CDN)	Font Awesome 6.4.0, tsparticles	Iconografía y fondo animado del login

2.2 Diagrama de arquitectura

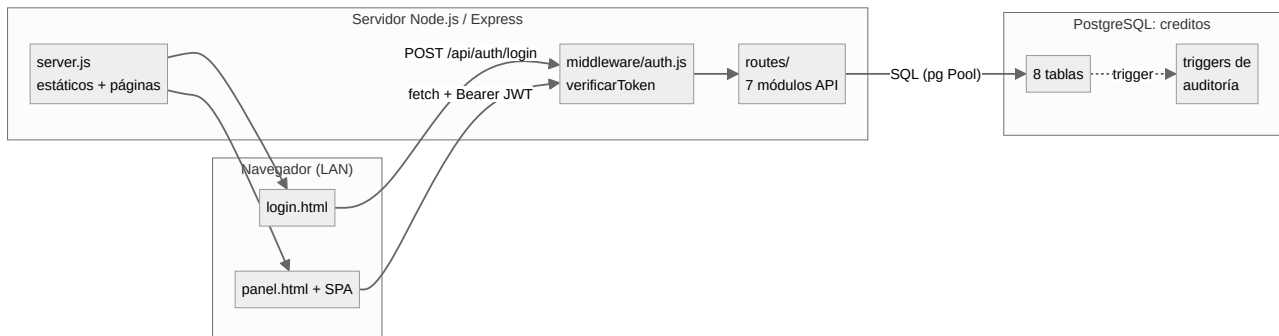


Figura 2-1. Vista general: navegador, servidor Express y base de datos

Dos decisiones arquitectónicas definen la forma actual del sistema:

- **Módulo genérico de tiendas (v6).** En lugar de tres bloques de código casi idénticos, un único router (`routes/tiendas.js`) y una única clase frontend (`TiendaApp` en `tienda-spa.js`) atienden a las tres tiendas mediante configuración (nombre de tabla, prefijos de ID, secciones). Las rutas históricas (`/api/tienda-caracas` , etc.) se conservan como alias de compatibilidad.
- **Seguridad centralizada (v6.1).** Todo endpoint de datos pasa por `verificarToken` , que no solo valida la firma del JWT sino que consulta la base de datos en cada petición: verifica que el usuario siga activo y que su `token_version` no haya sido incrementado (logout, cambio de contraseña o cambios administrativos invalidan sesiones previas al instante).

3. Estructura del proyecto

El proyecto reside en la estación de trabajo en `C:\Users\jhonny\Desktop\creditos`. A continuación se reproduce el árbol real (excluyendo `node_modules`), anotado con la función de cada elemento:

creditos/	
├─ server.js	Servidor Express (módulos, páginas, CORS, errores)
├─ package.json	Dependencias y scripts (start / dev)
├─ .env	Variables reales (NO versionable)
├─ .env.example	Plantilla documental de variables (sin valores)
├─ config/database.js	Pool de conexiones PostgreSQL compartido
├─ middleware/auth.js	verificarToken, soloAdmin, registrarAuditoria
├─ routes/	API REST (7 módulos)
│ └─ auth.js	Login, logout, diagnóstico de IP
│ └─ usuarios.js	CRUD usuarios, recuperación de clave, auditoría
│ └─ tiendas.js	CRUD genérico de tiendas + alias legacy
│ └─ bcv.js	Tasas BCV (actual, historial, por fecha)
│ └─ actividades.js	Agenda digital (tareas/eventos)
│ └─ reportes.js	Reportes operativos por tienda
│ └─ estadisticas.js	Estadísticas (depende de funciones SQL)
├─ public/	Frontend estático
│ └─ login.html	Página de acceso (pública)
│ └─ panel.html	SPA principal (post-login)
│ └─ tienda-caracas.html	LEGACY: página independiente de tienda
│ └─ reportes-caracas.html	LEGACY: reportes (JS referenciado ausente)
│ └─ assets/logo.png	
│ └─ css/	login.css, panel.css
│ └─ js/	LEGACY: tienda-caracas.css, tienda-maracaibo.css login.js, panel.js, tienda-spa.js (módulo genérico), tsparticles-config.js (fondo animado del login) LEGACY: reportes.js, tienda-caracas.js
├─ database/	
│ └─ init_usuarios.sql	Tablas de seguridad + usuario inicial (ejecutado)
│ └─ estadisticas.sql	4 funciones + 1 vista de estadísticas (NO ejecutado)
├─ facturas/	
│ └─ cargar_datos_tienda_caracas.sql	Carga inicial: 130 facturas
│ └─ facturas_con_tasa_BCV_CORREGIDO.xlsx	Fuente Excel de la carga
└─ informe tecnico/	Informes anteriores (PDF, versiones v5 y previas)

Observaciones sobre el árbol. (i) Los archivos marcados como LEGACY pertenecen a la arquitectura anterior a v6 y hoy duplican funciones del SPA; se recomienda retirarlos tras verificar que ningún usuario los tiene marcados en favoritos. (ii) `reportes-caracas.html` referencia `js/reportes-caracas.js`, archivo que no existe: la página legacy está rota. (iii) La carpeta `facturas/` contiene un archivo temporal de Excel (~\$....) eliminable. (iv) `database/estadisticas.sql` es la pieza faltante del módulo de estadísticas (ver [sección 9](#)).

4. Base de datos

La base de datos `creditos` (PostgreSQL, esquema `public`) contiene **8 tablas, 8 secuencias, 19 índices, 15 restricciones y 2 triggers**. La información de esta sección proviene del volcado de esquema (`pg_dump -s`) obtenido el 19 de julio de 2026, por lo que refleja el estado real de producción.

4.1 Tablas del sistema

Tabla 4-1. *Inventario de tablas*

Tabla	Columnas	Propósito
<code>usuarios</code>	11	Cuentas del sistema: credenciales, rol, tienda asignada, IP asignada y <code>token_version</code> para revocación de sesiones
<code>tienda_caracas</code>	75	Facturas a crédito de Caracas y sus 11 cuotas (estructura plana)
<code>tienda_maracay</code>	75	Idéntica estructura para Maracay (incluye trigger de <code>updated_at</code>)
<code>tienda_maracaibo</code>	74	Idéntica estructura para Maracaibo, excepto que carece de <code>created_at</code>
<code>actividades</code>	~10	Agenda digital: tareas/eventos con fecha, estado y tienda
<code>auditoria_usuario</code> <code>s</code>	7	Bitácora automática de altas, cambios y bajas de usuarios (alimentada por trigger)
<code>reset_tokens</code>	~6	Tokens de un solo uso para recuperación de contraseña
<code>sesiones</code>	9	Diseñada para rastrear sesiones activas (token, dispositivo, IP, expiración). Sin uso: ningún módulo la lee ni la escribe

4.1.1 Tabla `usuarios` (núcleo de seguridad)

<code>id</code>	<code>integer</code> PK (secuencia)	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	
<code>email</code>	<code>varchar(100)</code> NOT NULL UNIQUE	
<code>password</code>	<code>varchar(255)</code> NOT NULL	-- hash bcrypt (12 rondas)
<code>rol</code>	<code>varchar(20)</code> DEFAULT 'operador'	-- CHECK: administrador operador
<code>activo</code>	<code>boolean</code> DEFAULT true	
<code>ip_asignada</code>	<code>varchar(45)</code>	-- restricción de acceso por IP (nullable)
<code>tienda</code>	<code>varchar(20)</code>	-- alcance de tienda para operadores
<code>token_version</code>	<code>integer</code> NOT NULL DEFAULT 0	-- revocación de sesiones (v6.1)
<code>created_at / updated_at</code>	<code>timestamp</code> DEFAULT now()	

4.1.2 Tablas de tienda (estructura plana de 11 cuotas)

Cada tabla de tienda guarda una fila por factura a crédito, con los datos del cliente y la factura más un bloque repetido de **cinco campos por cada una de las 11 cuotas**: `cuota_N` (monto en bolívares), `ref_cuota_N` (referencia de pago), `fecha_cuota_N`, `tasa_cuota_N` (tasa BCV aplicada, `numeric(15,4)`) y `dolar_depositado_cuota_N` . Los campos financieros de cabecera son `monto_factura` ,

`monto_facturado_divisa`, `dolar_facturado`, `monto_pendiente`, `monto_depositados` y `deuda` (todos `numeric(15,2)`), más `cuotas` (entero, por defecto 11).

Nota de diseño. La estructura plana (55 columnas de cuotas en una sola tabla) es simple y rápida para el caso de uso — un número fijo de 11 cuotas — y evita *joins*. Su costo es la rigidez: cambiar el número de cuotas exigiría `ALTER TABLE`. La vista `v_resumen_cuotas_mes` (sección 4.3) fue diseñada precisamente para "desnormalizar" este esquema plano a efectos de análisis.

4.2 Índices, triggers y auditoría

Los 19 índices (todos *btree*) se concentran en las rutas de búsqueda reales del sistema:

- **Tiendas:** búsqueda por `cedula`, `nombre_apellido`, `nro_factura` y `fecha_factura`. Cobertura desigual: Caracas tiene índices en cédula, nombre y número de factura; Maracay añade `deuda` y `fecha_factura`; Maracaibo carece de índice en `nombre_apellido` y `deuda`.
- **Actividades:** cuatro índices sobre `fecha`, `estado`, `tienda` y el compuesto `(fecha, estado)` — la tabla mejor indexada del sistema.
- **Seguridad:** `auditoria_usuarios(usuario_id)` y `(created_at DESC)`; `sesiones(usuario_id)` y `sesiones(token)` — estos dos últimos sobre la tabla sin uso.

Tabla 4-2. Triggers instalados

Trigger	Tabla	Función
<code>trg_auditoria_usuarios</code>	<code>usuarios</code>	Tras <code>INSERT/UPDATE/DELETE</code> , registra en <code>auditoria_usuarios</code> la acción y el estado anterior/nuevo (JSONB). Auditoría a nivel de base de datos, independiente de la aplicación
<code>trigger_update_tienda_maracay</code>	<code>tienda_maracay</code>	Actualiza <code>updated_at</code> automáticamente en cada <code>UPDATE</code> . Existe solo en Maracay ; en Caracas y Maracaibo el campo queda congelado

4.3 Capa de estadísticas (definida pero no instalada)

El archivo `database/estadisticas.sql` (255 líneas) define una vista y cuatro funciones que constituyen la capa de analítica del sistema. **Ninguna existe en la base de datos:** el script nunca fue ejecutado. Es la causa raíz del estado del módulo de estadísticas (sección 9).

Tabla 4-3. Objetos SQL pendientes de instalación

Objeto	Tipo	Consumido por
<code>v_resumen_cuotas_mes</code>	Vista	Base de las 4 funciones; descompone las 11 cuotas por fila en filas individuales por cuota (CTE <code>cuotas_desglosadas</code>)

<code>f_estadisticas_periodo(fecha_ini, fecha_fin)</code>	Función	GET <code>/api/estadisticas/</code> y <code>/exportar</code>
<code>f_deudores_mes(mes, anio)</code>	Función	GET <code>/api/estadisticas/deudores</code>
<code>f_evolucion_pagos(anio)</code>	Función	GET <code>/api/estadisticas/evolucion</code>
<code>f_distribucion_pagos(fecha_ini, fecha_fin)</code>	Función	GET <code>/api/estadisticas/distribucion</code>

4.4 Scripts de inicialización y carga

Tabla 4-4. *Scripts SQL del repositorio*

Script	Función y estado
<code>database/init_usuarios.sql</code>	Crea <code>auditoria_usuarios</code> , <code>sesiones</code> , <code>reset_tokens</code> , sus índices, la función <code>registrar_auditoria()</code> y el primer usuario administrador. Ejecutado (los objetos existen en la BD)
<code>database/estadisticas.sql</code>	Vista + 4 funciones de estadísticas. No ejecutado
<code>facturas/cargar_datos_tienda_caracas.sql</code>	Carga inicial de 130 facturas reales de Caracas (130 <code>INSERT</code> generados desde el Excel fuente). Ejecutado una sola vez
<code>migracion-seguridad-v6.1.sql</code>	Agrega <code>token_version</code> a <code>usuarios</code> . Ya aplicada (la columna existe con <code>NOT NULL DEFAULT 0</code>)

5. API REST

El backend expone **37 endpoints REST** agrupados en 7 módulos bajo `/api/`, más 4 rutas de páginas HTML. Tras v6.1, **todos los endpoints de datos requieren JWT**; los únicos públicos son el login y los dos de recuperación de contraseña. Convención de respuestas: JSON; errores con `{ "error": "mensaje" }` sin filtrar detalles internos.

5.1 Autenticación — `/api/auth`

Tabla 5-1. *Endpoints de autenticación*

Método	Ruta	Acceso	Descripción
POST	<code>/login</code>	Público + rate limit	Valida credenciales, IP asignada y estado activo; emite JWT de 8 h con <code>tv</code> (token_version). Máx. 10 intentos fallidos por IP cada 15 min
POST	<code>/logout</code>	JWT	Incrementa <code>token_version</code> : invalida el token actual y cualquier sesión previa del usuario
GET	<code>/debug/ip</code>	JWT + administrador	Diagnóstico: muestra la IP detectada del cliente (útil para configurar <code>ip_asignada</code>)

5.2 Usuarios — `/api/usuarios`

Tabla 5-2. *Endpoints de usuarios*

Método	Ruta	Acceso	Descripción
GET	<code>/</code>	JWT + admin	Listado de usuarios
GET	<code>/:id</code>	JWT	Detalle de un usuario
POST	<code>/</code>	JWT + admin	Crear usuario (contraseña mín. 8 caracteres)
PUT	<code>/:id</code>	JWT	Editar usuario; si un admin cambia rol, estado, IP o tienda, se incrementa <code>token_version</code> (revoca sus sesiones)
DELETE	<code>/:id</code>	JWT + admin	Baja lógica (desactivación) con revocación de sesiones
PATCH	<code>/:id/reactivar</code>	JWT + admin	Reactivar un usuario dado de baja
PUT	<code>/:id/password</code>	JWT	Cambio de contraseña; revoca todas las sesiones previas
POST	<code>/recuperar-password</code>	Público	Genera token de recuperación (v6.1: ya no se devuelve en la respuesta)

POST	<code>/restablecer-password</code>	Público	Restablece contraseña con token válido; revoca sesiones
GET	<code>/perfil/me</code>	JWT	Perfil del usuario autenticado
GET	<code>/auditoria/:usuarioId</code>	JWT + admin	Historial de auditoría de un usuario
GET	<code>/estadisticas/resumen</code>	JWT + admin	Resumen de cuentas (totales, activos, por rol)

5.3 Tiendas — `/api/tiendas` (genérico v6)

Tabla 5-3. Endpoints del CRUD genérico de tiendas

Método	Ruta	Acceso	Descripción
GET	<code>/:tienda</code>	JWT + tienda propia	Listado completo de la tienda (caracas, maracay o maracaibo ; lista blanca en el router)
GET	<code>/:tienda/:id</code>	JWT + tienda propia	Detalle de una factura
POST	<code>/:tienda</code>	JWT + tienda propia	Crear factura (campos dinámicos validados contra la tabla)
PUT	<code>/:tienda/:id</code>	JWT + tienda propia	Actualizar factura / registrar cuotas
DELETE	<code>/:tienda/:id</code>	JWT + admin	Eliminar factura (solo administradores)

Compatibilidad: `/api/tienda-caracas`, `/api/tienda-maracay` y `/api/tienda-maracaibo` exponen el mismo CRUD (5 operaciones cada una, 15 endpoints alias) delegando al router genérico con idénticas validaciones de token y tienda. El parámetro `:tienda` nunca se interpola sin validar: una lista blanca interna lo traduce al nombre físico de la tabla, por lo que valores arbitrarios se rechazan con 400 y no hay vía de inyección SQL por ese canal.

5.4 Tasas BCV — `/api/bcv`

Tabla 5-4. Endpoints de tasas de cambio

Método	Ruta	Acceso	Descripción
GET	<code>/actual</code>	JWT	Tasa BCV vigente
GET	<code>/anterior</code>	JWT	Tasa del día anterior
GET	<code>/fechas</code>	JWT	Fechas con tasa registrada
GET	<code>/fecha/:fecha</code>	JWT	Tasa de una fecha específica
GET	<code>/historial/:year</code>	JWT	Historial anual de tasas

5.5 Actividades — `/api/actividades`

Tabla 5-5. Endpoints de la agenda digital

Método	Ruta	Acceso	Descripción
--------	------	--------	-------------

GET	/	JWT	Listado de actividades
GET	/:id	JWT	Detalle
POST	/	JWT	Crear actividad
PUT	/:id	JWT	Actualizar actividad
DELETE	/:id	JWT	Eliminar actividad
GET	/estadisticas/resumen	JWT	Resumen por estado (pendientes, completadas, etc.)

5.6 Reportes y estadísticas

Tabla 5-6. Endpoints de reportes y estadísticas

Método	Ruta	Acceso	Descripción
POST	/api/reportes/:tienda	JWT + tienda	Reporte operativo filtrado de la tienda (v6 unificó los bloques duplicados y habilitó Maracay, que no tenía endpoint)
GET	/api/estadisticas/	JWT	Estadísticas del período — requiere <code>f_estadisticas_periodo</code> (no instalada)
GET	/api/estadisticas/deudores	JWT	Deudores del mes — requiere <code>f_deudores_mes</code> (no instalada)
GET	/api/estadisticas/evolucion	JWT	Evolución anual de pagos — requiere <code>f_evolucion_pagos</code> (no instalada)
GET	/api/estadisticas/distribucion	JWT	Distribución de pagos — requiere <code>f_distribucion_pagos</code> (no instalada)
GET	/api/estadisticas/exportar	JWT	Exportación — misma dependencia no instalada. En los 5 casos el operador queda limitado a su tienda (se fuerza desde el token, no desde el cliente)

5.7 Rutas de páginas

Tabla 5-7. Páginas HTML servidas por Express

Ruta	Archivo	Estado
/	public/login.html	Operativa
/panel	public/panel.html	Operativa (SPA principal)
/tienda-caracas	public/tienda-caracas.html	Operativa pero LEGACY (anterior a v6)
/estadisticas	public/estadisticas.html	Rota: el archivo no existe; <code>sendFile</code> falla con error 500

6. Modelo de seguridad (v6.1)

La versión 6.1 es el resultado de una auditoría de seguridad completa sobre el sistema de autenticación y autorización. La auditoría identificó **6 hallazgos críticos, 6 altos y 10 medios** —entre ellos endpoints de tiendas y estadísticas totalmente abiertos, un token de recuperación de contraseña que se devolvía en la respuesta (toma de control de cuentas), un secreto JWT débil y la posibilidad de suplantar la restricción por IP con el encabezado `X-Forwarded-For` — y todos fueron corregidos en el paquete entregado e instalado. Esta sección documenta el modelo resultante.

6.1 Autenticación y revocación de sesiones

La autenticación es por **JWT firmado con HS256** (secreto de 96 caracteres hexadecimales generado criptográficamente, rotado en v6.1) con expiración de 8 horas. Lo distintivo del modelo es la **revocación inmediata**, resuelta sin tabla de sesiones mediante la columna `usuarios.token_version`:

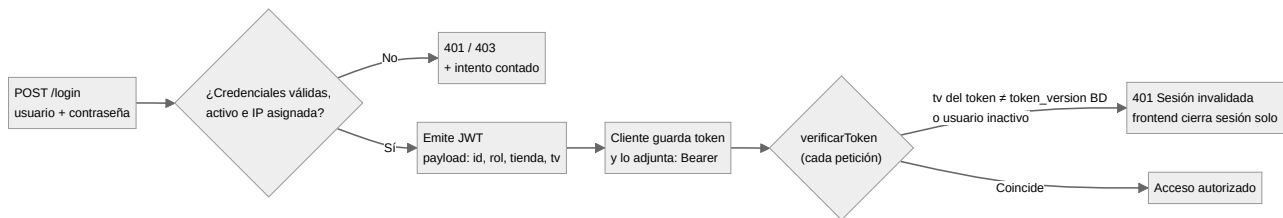


Figura 6-1. Flujo de autenticación y verificación con revocación por `token_version`

El middleware `verificarToken` consulta la base de datos en **cada petición** y compara el claim `tv` del token con el `token_version` actual del usuario. Cualquiera de estos eventos incrementa el contador e invalida al instante todos los tokens previos del usuario:

- Cierre de sesión (`POST /api/auth/logout`).
- Cambio o restablecimiento de contraseña.
- Que un administrador modifique el rol, el estado activo, la IP asignada o la tienda del usuario.
- La desactivación (baja lógica) del usuario.

Del lado cliente, un **interceptor global de fetch** en `panel.js` adjunta automáticamente el encabezado `Authorization` a toda llamada `/api/` y, al recibir un 401 por sesión invalidada, ejecuta el cierre de sesión local y redirige al login — resolviendo el problema de los "tokens perdidos" (sesiones huérfanas que antes permanecían abiertas hasta expirar).

6.2 Control de acceso por roles y tienda

Tabla 6-1. Matriz de autorización

Recurso / acción	Administrador	Operador
------------------	---------------	----------

Login / recuperar contraseña	Sí	Sí
Consultar su tienda (CRUD lectura/escritura)	Sí, las tres tiendas	Sí, solo su tienda asignada
Consultar otra tienda	Sí	No (403, validado en servidor desde el token, no desde el cliente)
Eliminar facturas	Sí	No (solo admin)
Gestión de usuarios (crear, editar, desactivar)	Sí	No
Auditoría y resumen de cuentas	Sí	No
Estadísticas y reportes	Cualquier tienda	Se fuerza a su tienda desde el token

6.3 Protecciones de red y aplicación

Tabla 6-2. Controles implementados en v6.1

Control	Implementación
Fuerza bruta en login	<code>express-rate-limit</code> : 10 intentos fallidos por IP cada 15 min; los éxitos no cuentan
Restricción por IP	La IP se toma de la conexión TCP real (<code>req.socket.remoteAddress</code>); solo se honran cabeceras <code>proxy</code> si <code>trust proxy</code> se habilita explícitamente. Inmune a suplantación por <code>X-Forwarded-For</code>
CORS	Lista blanca por variable de entorno <code>ALLOWED_ORIGINS</code> ; peticiones sin origen (mismo servidor) permitidas
Contraseñas	bcrypt 12 rondas; longitud mínima 8; nunca se registran en logs
Recuperación de contraseña	El token de recuperación ya no viaja en la respuesta de la API (pendiente canal de entrega: correo)
Inyección SQL	Consultas parametrizadas en todos los módulos; nombres de tabla por lista blanca
Fuga de errores	Las respuestas de error ya no incluyen <code>err.message</code> interno
Secreto JWT	Rotado a valor aleatorio de 96 hex; variable de entorno, fuera del código
Auditoría	Trigger en base de datos sobre <code>usuarios</code> (quién cambió qué, antes/después en JSONB) + endpoint de consulta para administradores

Pendiente reconocido (fuera del alcance de v6.1). El tráfico entre clientes y servidor circula en **HTTP plano** dentro de la LAN. Para uso estrictamente interno el riesgo es moderado, pero se recomienda evaluar HTTPS (certificado interno) o una VPN si el acceso desde otras redes se habilita en el futuro.

7. Frontend

El frontend es JavaScript puro sin proceso de compilación, servido como estático desde `public/`. Consta de dos superficies: el **login** (pública) y el **panel SPA** (post-autenticación), más las páginas legacy señaladas en la sección 3.

7.1 Login (`login.html` + `login.js` + `login.css`)

Página de acceso con fondo animado (`tsparticles`). `login.js` envía las credenciales a `/api/auth/login` y, ante éxito, persiste token y datos de usuario en `localStorage` y redirige a `/panel`. La corrección v6.1 resolvió una incompatibilidad de contrato: el cliente leía `data.user` mientras el backend enviaba `data.usuario`, lo que dejaba el perfil vacío tras el inicio de sesión.

7.2 Panel SPA (`panel.html` + `panel.js`, 3.033 líneas)

Aplicación de página única con navegación por secciones: panel principal, las tres tiendas, actividades, reportes, estadísticas y administración de usuarios. Responsabilidades de `panel.js`: router de secciones, interceptor global de `fetch` (adjunta token, cierra sesión ante 401), cierre de sesión con llamada a `/api/auth/logout` (con `keepalive` para sobrevivir a la recarga), módulos de BCV, actividades, reportes con exportación a PDF/Excel (jsPDF + SheetJS) y gráficos (ApexCharts).

7.3 Módulo genérico de tiendas (`tienda-spa.js`, 2.894 líneas)

La clase `TiendaApp` genera y opera toda la interfaz de una tienda a partir de una configuración declarativa (`TIENDAS_CONFIG`): clave, nombre, sección del menú, endpoints, prefijos de identificadores (`tc-`, `tm-`, `tmb-`) y estilos. Una sola clase sirve a las tres tiendas, eliminando ~1.200 líneas duplicadas. Características técnicas:

- **Mismos IDs y estilos que el diseño original:** la refactorización no alteró la experiencia de usuario.
- **Delegación de eventos** mediante atributos `data-action` (sin `onclick` en línea), lo que hace el HTML generado más seguro y mantenible.
- **Utilidades compartidas:** formato monetario venezolano (`80.075,55`), parseo numérico localizado, indicadores de carga y modales corporativos.
- **Autenticación integrada (v6.1):** todas sus peticiones pasan por un cliente `_apiFetch` que adjunta el JWT.

8. Refactorización v6: antes y después

La versión 6 eliminó la triplicación sistemática que el código de tiendas arrastraba (tres SPAs, tres bloques de rutas, reportes duplicados). El criterio rector fue **compatibilidad total hacia atrás**: mismos identificadores de elementos, mismas rutas de API (vía alias legacy) y misma apariencia.

Tabla 8-1. Métricas de la refactorización

Componente	Antes (v5)	Después (v6.1)	Variación
<code>server.js</code>	663 líneas	124 líneas	−81 % (endpoints de tienda extraídos a su módulo)
<code>panel.html</code>	1.631 líneas	832 líneas	−49 % (secciones de tienda reemplazadas por puntos de montaje)
<code>panel.js</code>	4.183 líneas	3.033 líneas	−27 % (reportes duplicados eliminados)
SPAs de tienda	3 archivos (Caracas, Maracay, Maracaibo)	1 archivo genérico (2.894 líneas)	3 → 1 (configuración en lugar de copias)
Rutas de tienda	3 bloques repetidos en <code>server.js</code>	1 router genérico + 3 alias	Validación única con lista blanca
Reportes por tienda	2 bloques duplicados (Maracay sin endpoint)	1 endpoint genérico para las 3 tiendas	Maracay habilitada por primera vez

Además de la consolidación, la refactorización corrigió **9 defectos funcionales**, entre ellos: la sección "Créditos" de Maracay que abría un marcador de "En desarrollo" en lugar del módulo, estilos ausentes para Maracay (la sección se mostraba sin formato), y la ausencia total del endpoint de reportes de Maracay. La verificación incluyó pruebas de humo con jsdom (montaje de las tres tiendas y delegación de eventos), pruebas funcionales de cálculo (deuda, cuotas, estados, filtros, paginación, conciliaciones) y **24/24 pruebas de backend** sobre el router genérico, incluida la comprobación de que la lista blanca de tablas bloquea intentos de inyección por el parámetro `:tienda`.

Impacto en mantenibilidad. Agregar una cuarta tienda al sistema requiere hoy: crear la tabla (misma estructura), añadir una entrada en la lista blanca del router y una entrada en TIENDAS_CONFIG. Antes de v6 implicaba copiar y adaptar miles de líneas en seis archivos distintos.

9. Estado real del módulo de estadísticas

El cruce de tres evidencias —el código de `estadisticas.js`, el volcado del esquema de la base de datos y el árbol de archivos— permite concluir con certeza que el módulo de estadísticas quedó **desplegado a medias** en algún momento de su historia:

Tabla 9-1. Las tres piezas del módulo y su estado

Pieza	Dónde debería estar	Estado real
API (<code>routes/estadisticas.js</code>)	backend Desplegada	Presente y asegurada (JWT + alcance por tienda), pero sus consultas fallan al ejecutarse
Funciones SQL (4) + vista (1)	Instaladas en la base <code>creditos</code>	Ausentes de la base de datos; existen solo como <code>script database/estadisticas.sql</code> nunca ejecutado
Página <code>estadisticas.html</code>	<code>public/estadisticas.html</code>	No existe; la ruta <code>/estadisticas</code> en <code>server.js</code> apunta a un archivo inexistente (error 500)

Consecuencia operativa. Cualquier llamada a `/api/estadisticas/*` termina en error de base de datos ("function does not exist"), y visitar `/estadisticas` produce un 500. Si nadie ha reportado fallas, simplemente indica que el módulo no se usa hoy.

Opciones de corrección

- **Activar el módulo:** ejecutar `psql -U postgres -d creditos -f database/estadisticas.sql`, crear la página `estadisticas.html` (o integrar la sección dentro del panel SPA) y probar los 5 endpoints. Esto habilita deudores del mes, evolución anual, distribución de pagos y exportación.
- **Retirar el módulo:** si las estadísticas no son necesarias, eliminar la ruta `/estadisticas` de `server.js` y el mount `/api/estadisticas`, reduciendo superficie de ataque.

La decisión depende del uso previsto; cualquiera de las dos deja el sistema consistente. Este informe recomienda decidirlo explícitamente y no dejar el estado intermedio actual.

10. Despliegue y operación

10.1 Topología actual

Tabla 10-1. Entorno de producción

Aspecto	Valor
Equipo	PC de escritorio Windows 11 (estación de trabajo del administrador)
Ruta del proyecto	C:\Users\jhonny\Desktop\creditos
Base de datos	PostgreSQL local (localhost:5432), base creditos
Arranque del servidor	Manual: ventana de terminal con node server.js
Acceso de usuarios	Navegadores de la LAN apuntando a la IP del equipo, puerto del servidor
Auto-reinicio ante fallos	No existe; si el proceso muere o la PC se reinicia, el servicio queda detenido
HTTPS	No (HTTP plano dentro de la LAN)
Respaldo de base de datos	No se evidenciaron tareas programadas de respaldo

10.2 Riesgos operativos del arranque manual

La dependencia de una ventana de terminal abierta concentra tres riesgos: **cierre accidental** de la ventana (detiene el servicio para toda la oficina), **reinicio del equipo** (actualizaciones de Windows, cortes eléctricos) sin levantamiento automático, y **ausencia de registros persistentes** (los logs mueren con la ventana). Se recomienda convertir el servidor en un servicio de Windows; dos caminos sencillos:

```
:: Opción A – PM2 con arranque automático
npm install -g pm2 pm2-windows-startup
pm2 start server.js --name creditos-ipsfa
pm2-startup install
pm2 save

:: Opción B – NSSM (servicio nativo de Windows)
nssm      install      CreditosIPSFA      "C:\Program      Files\nodejs\node.exe"
"C:\Users\jhonny\Desktop\creditos\server.js"
```

Como complemento se recomienda un **respaldo diario automatizado** de la base de datos, por ejemplo con una tarea programada que ejecute:

```
"C:\Program      Files\PostgreSQL\16\bin\pg_dump.exe"      -U      postgres      -d      creditos      -f
"D:\respaldos\creditos_%DATE%.sql"
```

10.3 Acceso desde otra red

Actualmente el sistema solo es alcanzable dentro de la red local. Habilitar acceso externo exigiría, como mínimo: IP fija o DNS dinámico, apertura controlada de puertos en el router/firewall, **HTTPS obligatorio** y preferiblemente una VPN en lugar de exposición directa. La restricción por IP asignada (v6.1) y la revocación de sesiones ya están preparadas para ese escenario, pero el canal cifrado es prerequisite: el JWT y las contraseñas no deben viajar sin cifrar fuera de la LAN.

11. Hallazgos y recomendaciones

Consolidación de todos los hallazgos del análisis (código, esquema de base de datos, árbol de archivos y despliegue), ordenados por prioridad. Los seis críticos y seis altos de seguridad detectados en la auditoría previa ya fueron corregidos en v6.1 y no se repiten aquí; esta tabla recoge lo que permanece abierto.

Tabla 11-1. Hallazgos abiertos, ordenados por prioridad

ID	Prioridad	Hallazgo	Recomendación
H-01	Alta	Funciones y vista de estadísticas no instaladas en la base de datos; los 5 endpoints de <code>/api/estadisticas</code> fallan	Ejecutar <code>database/estadisticas.sql</code> o retirar el módulo (ver sección 9)
H-02	Alta	Ruta <code>/estadisticas</code> sirve un HTML inexistente (error 500)	Crear la página, integrarla al panel o eliminar la ruta
H-03	Media	Servidor sin auto-arranque ni auto-reinicio (terminal manual)	Convertir en servicio de Windows (PM2 o NSSM, sección 10.2)
H-04	Media	Sin respaldos automáticos de la base de datos	Tarea programada diaria con <code>pg_dump</code> a ubicación segura
H-05	Media	Tabla <code>sesiones</code> (con 2 índices) sin uso por ningún módulo	Eliminarla, o adoptarla si algún día se desea "ver sesiones activas por dispositivo"
H-06	Media	<code>tienda_maracaibo</code> carece de la columna <code>created_at</code> (sus pares sí la tienen)	<code>ALTER TABLE tienda_maracaibo ADD COLUMN created_at timestamp DEFAULT CURRENT_TIMESTAMP;</code>
H-07	Media	Trigger de <code>updated_at</code> existe solo en Maracay; en las otras dos tiendas el campo no se actualiza	Replicar <code>update_tienda_maracay_updated_at()</code> para Caracas y Maracaibo
H-08	Baja	Cobertura de índices desigual entre tiendas (Maracaibo sin índice en <code>nombre_apellido</code> ni <code>deuda</code> ; Caracas sin <code>deuda</code> ni <code>fecha_factura</code>)	Homologar índices según los filtros más usados en cada módulo
H-09	Baja	Páginas legacy: <code>tienda-caracas.html</code> operativa pero redundante; <code>reportes-caracas.html</code> con JS referenciado inexistente (rota)	Retirar ambas junto con <code>tienda-caracas.js</code> , <code>reportes.js</code> y los CSS legacy, tras confirmar que nadie las usa
H-10	Baja	HTTP plano en la LAN	Evaluar certificado interno o VPN; prerequisite si se habilita acceso externo
H-11	Baja	Recuperación de contraseña sin canal de entrega (el token ya no se muestra por API, pero tampoco llega al usuario por correo)	Integrar envío por correo (SMTP institucional) o entrega asistida por el administrador
H-12	Info	Archivos temporales en el proyecto: <code>~\$facturas...xlsx</code> , <code>esquema2.sql</code> vacío, <code>arbol.txt</code>	Limpieza de repositorio

11.1 Plan sugerido

1. **Esta semana:** decidir el destino del módulo de estadísticas (H-01, H-02) y aplicar la limpieza menor (H-12).
2. **Próximo mantenimiento:** homologar esquema de tiendas (H-06, H-07, H-08) con una migración SQL única.
3. **Corto plazo:** servicio de Windows + respaldos automáticos (H-03, H-04) y retiro de legacy (H-09).
4. **Mediano plazo:** canal de correo para recuperación de contraseña (H-11) y, si se requiere acceso externo, HTTPS/VPN (H-10).

12. Anexos

Anexo A. Variables de entorno

El sistema se configura por `.env` (valores reales, nunca versionable) y `.env.example` (plantilla documental sin valores, segura de compartir). Claves reconocidas:

Tabla 12-1. Variables de entorno del sistema

Variable	Propósito
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Conexión a PostgreSQL
<code>JWT_SECRET</code>	Firma de tokens (96 hex, rotado en v6.1)
<code>JWT_EXPIRES_IN</code>	Vigencia del token (8 h)
<code>ALLOWED_ORIGINS</code>	Lista blanca CORS (vacía = solo mismo origen)
<code>PORT</code>	Puerto del servidor Express

Anexo B. Migración de seguridad v6.1 (ya aplicada)

```
ALTER TABLE usuarios
  ADD COLUMN IF NOT EXISTS token_version INTEGER NOT NULL DEFAULT 0;
```

Anexo C. Migraciones propuestas (pendientes)

```
-- H-06: homologar columna faltante en Maracaibo
ALTER TABLE tienda_maracaibo
  ADD COLUMN IF NOT EXISTS created_at timestamp DEFAULT CURRENT_TIMESTAMP;

-- H-07: replicar trigger de updated_at en Caracas y Maracaibo
CREATE TRIGGER trigger_update_tienda_caracas BEFORE UPDATE ON tienda_caracas
  FOR EACH ROW EXECUTE FUNCTION update_tienda_maracay_updated_at();
CREATE TRIGGER trigger_update_tienda_maracaibo BEFORE UPDATE ON tienda_maracaibo
  FOR EACH ROW EXECUTE FUNCTION update_tienda_maracay_updated_at();

-- H-08: homologar índices
CREATE INDEX IF NOT EXISTS idx_maracaibo_nombre ON tienda_maracaibo(nombre_apellido);
CREATE INDEX IF NOT EXISTS idx_maracaibo_deuda ON tienda_maracaibo(deuda);
CREATE INDEX IF NOT EXISTS idx_caracas_deuda ON tienda_caracas(deuda);
CREATE INDEX IF NOT EXISTS idx_caracas_fecha ON tienda_caracas(fecha_factura);
```

Anexo D. Convenciones de identificadores del frontend

Tabla 12-2. Prefijos por tienda en el módulo genérico

Tienda	Clave API	Prefijo de controles	Sufijo de sección
Caracas	caracas	tc-	(ninguno)
Maracay	maracay	tm-	-m
Maracaibo	maracaibo	tmb-	-mb

Anexo E. Fuentes del análisis

Este informe se elaboró exclusivamente a partir de evidencia primaria: 30 archivos fuente del sistema (código original y versiones v6/v6.1), el volcado de esquema de la base de datos (`esquema.sql` , 19-07-2026), los tres scripts SQL del repositorio, el árbol completo del proyecto y el informe técnico v5 previo como referencia histórica. Ningún dato de clientes fue utilizado ni se reproduce en este documento.

Fin del informe. Sistema de Créditos IPSFA v6.1 — Documento técnico interno generado el 19 de julio de 2026.